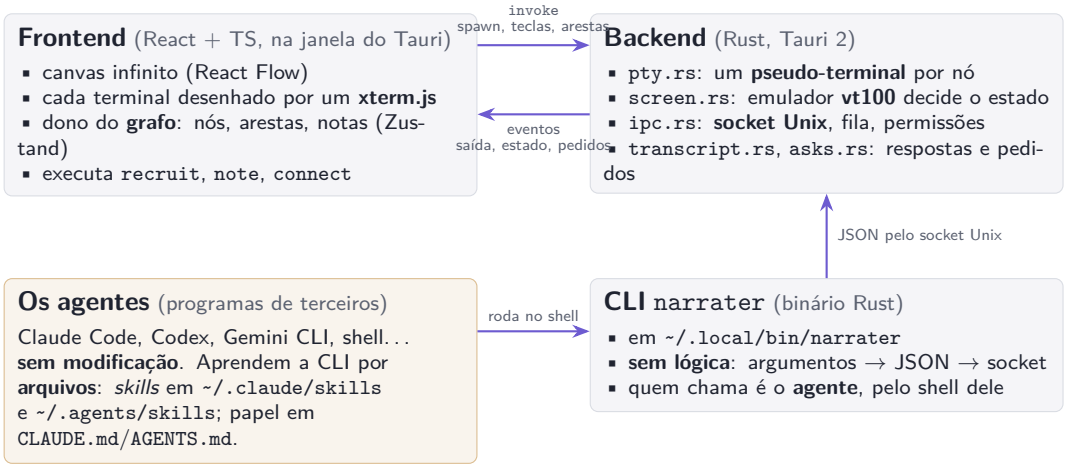
Como o NarraTer é feito

Arquitetura: três peças



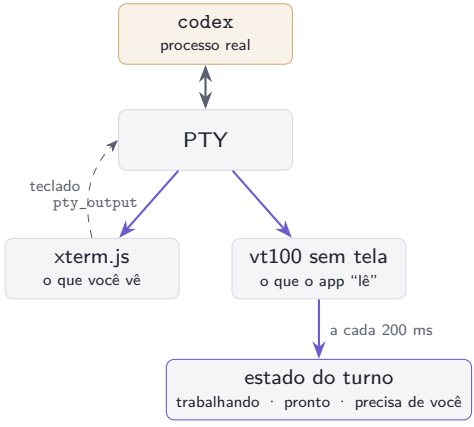
O que é um terminal no NarraTer

Quando um nó de terminal aparece no canvas, o frontend chama `pty_spawn` e o backend:

1. escreve o **papel** em `.narrater/roles/<id>/CLAUDE.md` e `AGENTS.md`, e inicia o agente nessa pasta, porque toda CLI lê esses arquivos sozinha;
2. abre um **pseudo-terminal** (PTY) e roda o programa nele, dentro de uma sessão `tmux` própria quando a sessão é persistente (sobrevive ao app fechar);
3. injeta a **identidade** do terminal no ambiente:

```
NARRATER_SOCKET=/tmp/narrater-<pid>/narrater.sock
NARRATER_ID=<id do nó>   NARRATER_LABEL=Ana
NARRATER_TOKEN=<segredo deste terminal>
```

4. lê a saída do PTY e manda **para dois lugares**.



O agente nunca sabe que está no NarraTer: para ele, é um terminal comum.

O que acontece num `narrater ask`

O agente líder roda, no shell dele:

```
narrater ask "Ana" "Rode os testes e me diga o resultado"
```

A CLI só lê o ambiente e manda ao socket:

```
{"from": "<NARRATER_ID>", "token": "<NARRATER_TOKEN>",
 "mode": "ask", "to": "Ana", "msg": "Rode os testes..."}
```

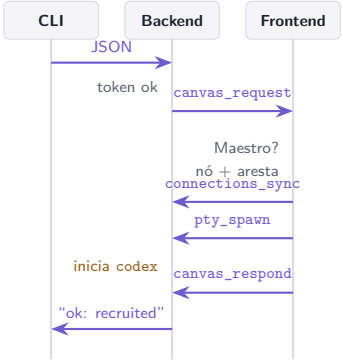
O socket fica numa pasta só do usuário (permissões 0700/0600). O *token* prova que o pedido veio mesmo daquele terminal: o nome que a CLI manda nunca é aceito sem conferência.

1. **Autentica**: o *token* bate com o do terminal `from`?
2. **Autoriza**: existe aresta entre o chamador e "Ana"? (`resolve_target`). Sem aresta, erro.
3. **Registra** o pedido em `asks.jsonl` (sobrevive a reinício) e **anota o tamanho** do *transcript* da Ana, para achar a resposta depois.
4. **Enfileira** `[narrater from Narrador]: Rode os testes...`
5. **Entrega** quando a Ana fica ociosa: *digita* o texto no PTY dela (colagem entre colchetes se o programa aceitar, senão texto + Enter após 300 ms).
6. **Espera o fim do turno**: tela (mudou? rodapé "esc to interrupt"? título da janela?) e *transcript* (`system/turn_duration`).
7. **Responde**: o último texto da Ana no *transcript*, ou a tela se não houver. A CLI imprime, e o líder lê como a saída de qualquer comando.

O que acontece num `narrater recruit`

```
narrater recruit "Ana" --preset Codex --role Developer
```

1. A **CLI** monta o JSON `{mode: canvas, action: recruit, name, preset, role}` e manda ao socket.
2. O **backend** autentica o *token*. Como o grafo pertence ao frontend, ele emite `canvas_request` e espera a resposta.
3. O **frontend** (`managerCommands.ts`) confere se o chamador está em **modo Maestro** (senão, erro), resolve o *preset* (o comando do Codex) e o papel, e cria o **nó** num lugar livre ao lado do líder, com a **aresta** líder → Ana.
4. A mudança no grafo dispara duas coisas: `connections_sync` (o backend recebe a nova tabela de permissões) e `pty_spawn` (o terminal da Ana nasce, como no slide anterior).
5. O frontend devolve `ok: recruited 'Ana' ..., connected to you`, que volta pela CLI ao líder.



Em resumo: recrutar não cria um "agente" abstrato. Cria um nó, abre um processo real do Codex num pseudo-terminal, entrega as instruções por arquivo e uma aresta que dá permissão de conversar.

Como se usa

Como o NarraTer monta uma orquestração hoje

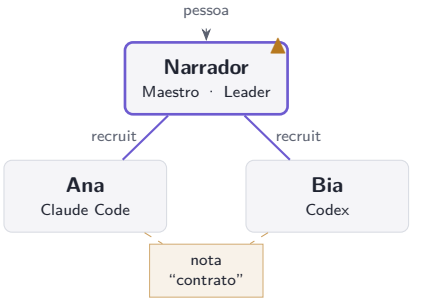
1. **A pessoa escolhe quem orquestra:** liga o **modo Maestro** num agente. Com o papel *Leader*, ele só delega, não edita código.
2. **O líder monta a equipe sozinho.**

```
narrater recruit "Ana" --preset "Claude Code" --role Dev
narrater recruit "Bia" --preset Codex --role Dev
```

   Cada um vira um terminal real, já ligado ao líder: **a aresta é a permissão** de conversar.
3. **Delega em paralelo.**

```
narrater ask --batch '{"Ana":"...","Bia":"..."}' --no-wait
narrater wait
```

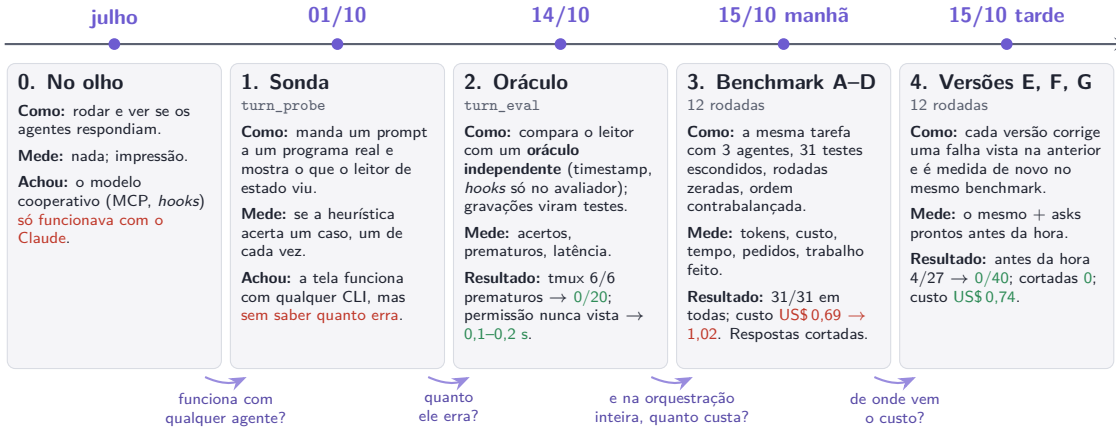
   O pedido é **digitado** no terminal do alvo quando ele fica ocioso.
4. **Recebe as respostas:** o NarraTer percebe o fim do turno e devolve a resposta final.



Instruções chegam por arquivo, que qualquer CLI lê: *skills* e papéis como `CLAUDE.md`/`AGENTS.md`. Nenhum agente precisa de plugin ou protocolo próprio.

Como foi testado

Como os testes evoluíram



Cada etapa nasceu de uma pergunta que a anterior não respondia. **Próxima:** comparar com um **agente único** (direto e em loop) na mesma tarefa, para responder quando a orquestração compensa.